TYPE MASTER

CS23531 – WEB PROGRAMMING
MINI PROJECT REPORT

Submitted by

AKASH T S – 241001011

Of

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**TYPE MASTER**" is the Bonafide work of **AKASH T S (241001011)** who carried out the work under my supervision.

SIGNATURE
Dr.P. Neelaveni
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

SIGNATURE
Ms. Janani M
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam,
Chennai - 602105

This project is submitted for CS23531 – Web Programming held on _____

INTERNAL EXAMINAR **EXTERNAL EXAMINAR**

ABSTRACT:

TypeMaster is a browser-based Typing Speed Tester built entirely with HTML5, CSS3, and JavaScript — no server or database required. It presents passages at three difficulty levels (Easy, Medium, Hard) and three durations (30, 60 or 120 seconds), and measures a user's typing speed and accuracy in real time. The dashboard displays live Words Per Minute (WPM), Characters Per Minute (CPM), accuracy percentage, error count and a countdown timer. Characters are highlighted green for correct and red for incorrect keystrokes. When the timer expires, a results overlay shows the final statistics and categorises the typist as Beginner, Intermediate, Advanced or Expert. Personal-best scores are saved to the browser's localStorage so progress is tracked across sessions. A dark/light theme toggle and responsive layout make TypeMaster comfortable on any device. By giving students and professionals an immediate, gamified measure of their typing speed, TypeMaster encourages deliberate practice and measurable improvement.

- AKASH TS

TABLE OF CONTENTS

S.No	Title	Page No
1.1	Introduction	4
1.2	Purpose	4
1.3	Scope of the Project	4
1.4	Software Requirement Specification	5
2.1	Use Case Diagram	8
2.2	Entity-Relationship Diagram	8
2.3	Data Flow Diagram	9
3	Module Description	10
4	Implementation	10
5	Conclusion	19
6	References	19

TYPEMASTER

1.1 INTRODUCTION

Fast, accurate typing is an essential skill for students, developers and professionals working on computers every day. Although many free typing tests exist online, they either require an account, display advertisements, or send keystroke data to a server — raising privacy concerns and adding unnecessary complexity.

TypeMaster is a self-contained, offline-capable web application that runs entirely in the browser. Users choose a difficulty level and a test duration, then begin typing the displayed passage. Live statistics — WPM, CPM, accuracy, errors and remaining time — update with every keystroke. When the timer expires, a results overlay presents the final score, a skill-level badge (Beginner through Expert), and a personal-best comparison. The project uses HTML5 for structure, CSS3 (custom properties, grid, animations) for styling, and a single vanilla JavaScript file for all logic — no frameworks, no bundler, no server.

1.2 PURPOSE

The purpose of TypeMaster is to give any user a fast, private, and accurate way to measure and improve their typing speed. It aims to:

- Present readable passages at three difficulty levels so beginners and expert typists both find an appropriate challenge.
- Track WPM and CPM in real time so users see their progress as they type, not just at the end.
- Provide a permanent personal-best record (stored in localStorage) so improvement is visible across sessions.
- Run entirely in the browser without a server, database, or user account, ensuring privacy and instant availability.
- Classify users into four skill levels — Beginner, Intermediate, Advanced and Expert — to give context to raw WPM scores.
- Offer a distraction-free interface with a dark/light theme toggle suitable for long practice sessions.

1.3 SCOPE OF THE PROJECT

TypeMaster targets individual users who want to measure or improve their typing on any modern browser. Its scope covers:

- **Difficulty selection:** Easy, Medium and Hard passage banks. Easy uses short, simple sentences; Medium uses complex prose; Hard uses numbers, symbols and punctuation.
- **Duration selection:** 30-second, 60-second and 120-second tests. The timer starts on the user's first keystroke.
- **Live statistics:** Words Per Minute, Characters Per Minute, accuracy percentage, error count and remaining time update on every keystroke.
- **Character-level feedback:** Correct characters are highlighted green and incorrect characters red; the current cursor position blinks.
- **Results overlay:** Shows final WPM, CPM, accuracy, errors, characters typed, skill-level badge and a motivational message.
- **Personal best:** The highest WPM and its accuracy are stored in localStorage and displayed in the toolbar and on the results overlay.

- **Theme:** A toggle switches between light and dark mode; the preference is persisted in localStorage.
- **Infinite text:** When the user nears the end of a passage, a new one is silently appended so fast typists never run out of text.

The project does not include user accounts, server-side leaderboards, multiplayer modes, or mobile-keyboard optimisation. These can be added in future versions.

1.4 SOFTWARE REQUIREMENTS OF THE PROJECT

Overall Description

TypeMaster lets any user measure and improve their typing speed and accuracy directly in a web browser. All logic runs client-side: the HTML file defines the structure, the CSS file defines the visual design, and the JavaScript file drives every interactive feature — passages, statistics, timer, results and themes.

Product Perspective

The system is a single-page web application with no server tier:

- **Presentation layer:** index.html — semantic markup, accessible ARIA attributes, nav, stats grid, typing area, results overlay, footer.
- **Style layer:** style.css — CSS custom properties for light/dark themes, responsive grid, animations (pulse, blink, pop).
- **Logic layer:** script.js — passage bank, state machine, statistics engine, render loop, timer, personal-best management, event handlers.

Hardware Requirements:

• Processor: Intel i3 or higher

• RAM: 4 GB or higher

• Hard Disk: 100 MB free space (for project files and browser cache)

• OS: Windows 8, 10, 11 / macOS / Linux

Software Requirements:

• Browser: Google Chrome, Mozilla Firefox, Microsoft Edge or Safari (any modern version)

• Frontend: HTML5, CSS3, JavaScript (ES2017+, strict mode)

• Fonts: Google Fonts – Poppins (UI) and JetBrains Mono (passage display)

• No backend, no database, no build tool required

Product Functionality

a) Difficulty Selection: user picks Easy, Medium or Hard; a new passage from the corresponding bank is loaded.

b) Duration Selection: user picks 30 s, 60 s or 120 s; the countdown is reset and the test restarts.

c) Live Statistics: WPM, CPM, accuracy, errors and time remaining update in real time on each input event.

d) Character Highlighting: correct = green, incorrect = red, current = teal blink. The passage auto-scrolls to keep the cursor visible.

e) Progress Bar: a gradient bar fills from left to right as elapsed time increases.

f) Results Overlay: on time-up, shows WPM, CPM, accuracy, errors, characters typed, skill badge and a motivational message.

g) Personal Best: if the current WPM beats the stored best, the new record is saved to localStorage and flagged in the overlay.

h) Restart / New Passage: restart re-runs the same passage; new passage loads a different one. Esc key also restarts.

i) Theme Toggle: switches the CSS data-theme attribute between 'light' and 'dark'; preference is persisted.

j) Anti-Cheat: paste and drop events are prevented so every score reflects real typing.

Assumptions and Dependencies

- A modern browser with JavaScript enabled is required; the app does not work with JS disabled.
- Google Fonts (Poppins and JetBrains Mono) load from the CDN; the app still functions with system fonts if offline.
- localStorage must be available for personal-best storage and theme persistence; the app degrades gracefully if it is blocked.
- Users are assumed to type on a physical keyboard; mobile keyboard use is functional but not optimised.

Functional Requirements

1. Passage Management Module (PMM)

- The PASSAGES object holds four passages each for Easy, Medium and Hard difficulty levels.
- A new passage is chosen at random, avoiding repetition of the immediately preceding passage.
- When fewer than 40 characters remain untyped, a new passage is silently appended.

2. Statistics Module (SM)

- WPM = (correct characters / 5) / elapsed minutes.
- CPM = total characters typed / elapsed minutes.
- Accuracy = (correct characters / total characters typed) × 100.
- A minimum of 2 seconds of elapsed time is used in division to avoid infinity on the first keystroke.

3. Timer Module (TM)

- The timer starts on the first keystroke, not on page load.
- The interval fires every 100 ms; statistics and the progress bar update on each tick.
- When elapsed time reaches the selected duration, the test is finalised automatically.

4. Results Module (RM)

- LEVELS array maps WPM thresholds to Beginner, Intermediate, Advanced and Expert labels.
- A motivational message is selected based on WPM range; a low-accuracy addendum is appended if accuracy < 90%.
- Personal best is loaded from and saved to localStorage under the key 'typemaster-best'.

5. Theme Module (ThM)

- setTheme() writes the data-theme attribute on <html> and persists the choice in localStorage.
- On initialisation, the saved theme is loaded; if none is saved, the OS preference (prefers-color-scheme) is used.

2. SYSTEM FLOW DIAGRAMS

2.1 USE CASE DIAGRAMS

2.2 ENTITY RELATIONSHIP DIAGRAMS

2.3 DATA FLOW DIAGRAM

3. MODULE DESCRIPTION

- **Passage Module (script.js – PASSAGES, randomPassage, extendTextIfNeeded):** holds the passage bank for all three difficulty levels, selects a passage at random (without immediate repetition) and silently appends a new one when the user is within 40 characters of the end.
- **Statistics Module (script.js – getStats, updateStats):** computes WPM, CPM, accuracy and error count from the current typed string and elapsed time, and pushes the values to the five stat tiles and the progress bar every 100 ms.
- **Render Module (script.js – render):** rebuilds the passage display on every keystroke by creating a per character, assigning class 'correct', 'incorrect' or 'current', and auto-scrolling to the cursor position.
- **Timer Module (script.js – startTimer, finishTest, resetTest):** starts the countdown on the first keystroke, fires every 100 ms, calls finishTest when the duration is reached, and resets all state variables and the hidden textarea for a fresh test.
- **Results Module (script.js – showResults, levelFor, messageFor, loadBest, saveBest):** populates the results overlay with the final statistics, selects the skill-level badge and motivational message, and updates the personal-best record in localStorage if the current WPM is a new high.
- **Theme Module (script.js – setTheme, init):** applies the selected colour theme via the data-theme attribute on <html>, persists the choice in localStorage, and reads the OS colour-scheme preference on first load.
- **Stylesheet (style.css):** defines all colours as CSS custom properties on :root for light mode and overrides them for dark mode under [data-theme='dark']; provides the responsive grid, stat tiles with colour-coded top borders, passage display, animations and the results overlay.
- **HTML Structure (index.html):** sticky navbar, hero section, practice card (toolbar, stats grid, progress bar, status line, passage display, hidden textarea, action buttons), About section with instructions and speed levels, results overlay, and footer.

4. IMPLEMENTATION - 4.1 SCREENSHOTS

4.2 PROJECT STRUCTURE

TypeMaster consists of three files. index.html is the single HTML page containing the full application structure: the sticky navbar with a theme toggle, the hero section, the practice card (toolbar with difficulty/duration chips, five stat tiles, a progress bar, the passage display, a hidden textarea for keyboard capture, and action buttons), an About section, a results overlay, and the footer. style.css defines every colour as a CSS custom property on :root and overrides the palette under [data-theme='dark'] for dark mode; it uses CSS Grid for the stats and result panels, border-radius and backdrop-filter for the glass-card aesthetic, and @keyframes for the cursor blink, live-dot pulse and overlay pop animation. script.js contains the entire application logic: the passage bank (PASSAGES), a shared state object, the statistics engine, the DOM render loop, the timer, the results and personal-best functions, and all event handlers. No external libraries or build tools are used.

4.3 CODE

1. PASSAGE BANK AND RANDOM SELECTION (script.js)

```
const PASSAGES = {
  easy: [
    "The sun is warm and the sky is blue. I like to walk in the park with my dog.",
    "My cat sleeps on the soft bed all day. She wakes up when it is time to eat.",
    // … two more easy passages
  ],
  medium: [
    "Good habits are built one day at a time. Practice every morning, track your
progress…",
    // … three more medium passages
  ],
  hard: [
    "In 2024, researchers analyzed 1,250 datasets; surprisingly, only 37% produced
reproducible results.",
    // … three more hard passages
  ]
};

function randomPassage() {
  const list = PASSAGES[state.difficulty];
  let index;
  // Keep picking until we get a passage different from the last one
  do { index = Math.floor(Math.random() * list.length); }
  while (list.length > 1 && index === state.lastIndex);
  state.lastIndex = index;
  return list[index];
}

// Silently appends a new passage when fewer than 40 characters remain
function extendTextIfNeeded() {
  if (state.text.length - state.typed.length < 40) {
    state.text += " " + randomPassage();
  }
}
```

2. STATISTICS ENGINE (script.js)

```
function getStats() {
  const typed = state.typed;
  let correct = 0;
  for (let i = 0; i < typed.length; i++) {
```

```
    if (typed[i] === state.text[i]) correct++;
  }
  let elapsed = 0;
  if (state.finished) elapsed = state.duration;
  else if (state.startTime)
    elapsed = Math.min((Date.now() - state.startTime) / 1000, state.duration);

  const minutes = Math.max(elapsed, 2) / 60; // prevents huge WPM in first second
  const safe = (n) => (Number.isFinite(n) ? Math.round(n) : 0);
  return {
    typed: typed.length,
    errors: typed.length - correct,
    wpm:  typed.length ? safe(correct / 5 / minutes) : 0,
    cpm:  typed.length ? safe(typed.length / minutes) : 0,
    accuracy: typed.length ? safe((correct / typed.length) * 100) : 100,
    elapsed,
    timeLeft: Math.max(state.duration - elapsed, 0)
  };
}
```

3. CHARACTER RENDER LOOP (script.js)

```
function render() {
  const fragment = document.createDocumentFragment();
  for (let i = 0; i < state.text.length; i++) {
    const span = document.createElement("span");
    span.textContent = state.text[i];
    if (i < state.typed.length) {
      // Already typed: correct = green, incorrect = red
      span.className = state.typed[i] === state.text[i] ? "correct" : "incorrect";
    } else if (i === state.typed.length && !state.finished) {
      span.className = "current"; // blinking cursor position
    }
    fragment.appendChild(span);
  }
  el.display.replaceChildren(fragment);
  // Auto-scroll so the current character is always visible
  const current = el.display.querySelector(".current");
  el.display.scrollTop = current ? Math.max(current.offsetTop - 70, 0) : 0;
}
```

4. TIMER AND TEST FLOW (script.js)

```
function startTimer() {
  state.running   = true;
  state.startTime = Date.now();
  setStatus("live");
  // Fire every 100 ms to keep the stats and progress bar smooth
  state.timerId = setInterval(() => {
    updateStats();
    if (Date.now() - state.startTime >= state.duration * 1000) finishTest();
  }, 100);
}

function finishTest() {
  clearInterval(state.timerId);
  state.running  = false;
  state.finished = true;
  el.input.disabled = true; // prevent further typing
  setStatus("done");
  render();
  updateStats();
  showResults();
}

function resetTest(newPassage = false) {
  clearInterval(state.timerId);
  Object.assign(state, {
    typed: "", startTime: null, timerId: null, running: false, finished: false
  });
  if (newPassage || !state.text) state.text = randomPassage();
```

```
  el.input.value    = "";
  el.input.disabled = false;
  el.overlay.classList.add("hidden");
  setStatus("idle");
  render();
  updateStats();
  el.input.focus();
}
```

5. RESULTS AND PERSONAL BEST (script.js)

```
const LEVELS = [
  { min: 80, name: "Expert" },
  { min: 50, name: "Advanced" },
  { min: 30, name: "Intermediate" },
  { min: 0,  name: "Beginner" }
];

function levelFor(wpm) {
  return LEVELS.find((level) => wpm >= level.min).name;
}

function showResults() {
  const s = getStats();
  el.rWpm.textContent      = s.wpm;
  el.rCpm.textContent      = s.cpm;
  el.rAccuracy.textContent = s.accuracy + "%";
  el.rErrors.textContent   = s.errors;
  el.rChars.textContent    = s.typed;
  el.rCategory.textContent = levelFor(s.wpm);
  el.rMessage.textContent  = s.typed
    ? messageFor(s.wpm, s.accuracy)
    : "No typing detected. Give it another go!";

  const best = loadBest();
  if (s.typed > 0 && s.wpm > (best ? best.wpm : 0)) {
    saveBest({ wpm: s.wpm, accuracy: s.accuracy });
    el.rBest.textContent = "New personal best!"; // highlight achievement
  } else {
    el.rBest.textContent = best ? "Personal best: " + best.wpm + " WPM" : "";
  }
  renderBest();
  el.overlay.classList.remove("hidden");
  el.tryAgain.focus(); // keyboard-accessible by default
}

function loadBest() {
  try { return JSON.parse(localStorage.getItem("typemaster-best")); }
  catch (error) { return null; }
}

function saveBest(best) {
  try { localStorage.setItem("typemaster-best", JSON.stringify(best)); }
  catch (error) { /* storage unavailable, ignore */ }
}
```

6. THEME TOGGLE (script.js)

```
function setTheme(theme) {
  document.documentElement.dataset.theme = theme; // applies CSS overrides
  el.theme.textContent = theme === "dark" ? "□" : "□";
  try { localStorage.setItem("typemaster-theme", theme); } catch (error) {}
}

el.theme.addEventListener("click", () => {
  setTheme(document.documentElement.dataset.theme === "dark" ? "light" : "dark");
});

// Initialise theme from localStorage or OS preference
(function init() {
```

```
  let saved = null;
  try { saved = localStorage.getItem("typemaster-theme"); } catch (error) {}
  setTheme(saved || (window.matchMedia("(prefers-color-scheme: dark)").matches ?
"dark" : "light"));
  renderBest();
  resetTest(true); // load the first passage
})();
```

7. EVENT LISTENERS (script.js)

```
// Main input handler — fires on every keystroke
el.input.addEventListener("input", () => {
  if (state.finished) { el.input.value = state.typed; return; }
  const value = el.input.value.replace(/\n/g, " ");
  state.typed  = value.slice(0, state.text.length);
  el.input.value = state.typed;
  if (!state.running && state.typed.length > 0) startTimer(); // start on first key
  extendTextIfNeeded();
  render();
  updateStats();
});

// Block paste and drop to ensure scores reflect real typing
["paste", "drop"].forEach((type) =>
  el.input.addEventListener(type, (e) => e.preventDefault())
);
// Block Enter to prevent accidental line breaks
el.input.addEventListener("keydown", (e) => {
  if (e.key === "Enter") e.preventDefault();
});

// Keyboard shortcuts
document.addEventListener("keydown", (e) => {
  if (e.key === "Escape") resetTest(false); // restart with same passage
});
```

8. HTML STRUCTURE (index.html — key sections)

```
<!-- Sticky navbar with brand, nav links and theme toggle -->
<nav class="navbar">
  <a class="brand" href="#home">TypeMaster</a>
  <div class="nav-links">
    <a href="#home">Home</a>
    <a href="#practice">Practice</a>
    <a href="#about">About</a>
    <button id="themeToggle" class="icon-btn" aria-label="Toggle dark
mode">□</button>
  </div>
</nav>

<!-- Practice card: toolbar, stats, progress, passage, actions -->
<section id="practice" class="card" aria-label="Typing test">
  <div class="toolbar">
    <div class="group" id="difficultyGroup" role="group" aria-label="Difficulty">
      <button class="chip active" data-difficulty="easy">Easy</button>
      <button class="chip" data-difficulty="medium">Medium</button>
      <button class="chip" data-difficulty="hard">Hard</button>
    </div>
    <div class="group" id="durationGroup" role="group" aria-label="Duration">
      <button class="chip" data-duration="30">30s</button>
      <button class="chip active" data-duration="60">60s</button>
      <button class="chip" data-duration="120">120s</button>
    </div>
    <span class="best" id="bestScore">Personal best: none yet</span>
  </div>
  <!-- Stats grid -->
  <div class="stats">
    <div class="stat"><span>⚡ WPM</span><strong id="wpm">0</strong></div>
    <div class="stat"><span>□ CPM</span><strong id="cpm">0</strong></div>
    <div class="stat"><span>□ Accuracy</span><strong
id="accuracy">100%</strong></div>
```

```
    <div class="stat"><span>⏱ Time left</span><strong
id="timeLeft">60s</strong></div>
    <div class="stat"><span>✗ Errors</span><strong id="errors">0</strong></div>
  </div>
  <!-- The passage display and the hidden textarea that captures typing -->
  <div id="textDisplay" class="text-display" aria-label="Passage to type"></div>
  <textarea id="typingInput" class="hidden-input" autocomplete="off"
    autocapitalize="off" autocorrect="off" spellcheck="false"></textarea>
</section>
```

9. CSS DESIGN TOKENS (style.css — custom properties)

```
:root {
  --bg1:     #ccfbf1;  /* teal gradient origin */
  --bg2:     #e0e7ff;  /* indigo gradient origin */
  --card:    rgba(255,255,255,.9); /* glass-card surface */
  --text:    #0f172a;  /* near-black for readability */
  --muted:   #64748b;  /* secondary labels */
  --primary: #0d9488;  /* teal accent */
  --good:    #15803d;  --good-bg: rgba(34,197,94,.16);   /* correct char */
  --bad:     #dc2626;  --bad-bg:  rgba(239,68,68,.16);   /* incorrect char */
}
:root[data-theme="dark"] {
  --bg1:  #042f2e;  --bg2:  #1e1b4b;
  --card: rgba(15,23,42,.9);  --text: #e2e8f0;
  --good: #4ade80;  --bad:  #f87171;
  /* All other tokens also overridden for full dark-mode support */
}
```

5. CONCLUSION

TypeMaster demonstrates that a fully-featured, privacy-respecting web application can be built with nothing more than HTML, CSS and vanilla JavaScript. It delivers real-time WPM and CPM measurement, character-level feedback, adaptive passage extension, personal-best tracking and a polished dark/light theme — all without a server, a database, a framework or a build tool.

The project gave me practical experience in DOM manipulation at scale (replacing hundreds of span elements on every keystroke without perceptible lag), CSS custom properties and theming, responsive grid layout, localStorage with graceful degradation, and writing a small but complete state machine in strict-mode JavaScript. It also reinforced the value of accessibility — ARIA labels, focus management on the results overlay, and keyboard navigation via Esc and Enter — ensuring the application is usable for all typists.

Future improvements could include a server-side global leaderboard, multiplayer race mode, custom passage upload, per-key error heatmap to highlight weak fingers, mobile-keyboard optimisation and an optional metronome for rhythm training.

6. REFERENCES

- MDN Web Docs (DOM, localStorage, CSS custom properties, ARIA), developer.mozilla.org
- Google Fonts – Poppins and JetBrains Mono, fonts.google.com
- CSS-Tricks – A Complete Guide to CSS Grid, css-tricks.com
- W3C Web Accessibility Initiative – ARIA Authoring Practices Guide, w3.org/WAI
- Visual Studio Code Documentation, code.visualstudio.com
- ECMAScript 2017 Language Specification (strict mode, async/await baseline), ecma-international.org